Apunte teórico - Lab. Algoritmos y Estructura de Datos (parte 4) – Python

ÍNDICE

1. Introducción a Python
2. Variables y tipos de datos
3. Entrada y salida de datos
4. Operadores aritméticos
5. Operadores de comparación y lógicos
6. Condicionales
7. Bucles (while, for)
8. Funciones
9. Strings
10. Listas
11. Manejo de archivos
12. Organización y buenas prácticas

---

1. INTRODUCCIÓN A PYTHON

¿Qué es Python?

Python es un lenguaje de programación que nos permite escribir instrucciones para que una computadora realice determinadas tareas.

Al igual que vimos con C, programar consiste en indicarle a la computadora qué debe hacer paso a paso.

La diferencia es que Python tiene una sintaxis más sencilla y utiliza menos símbolos para expresar las instrucciones.

Por ejemplo, en C para mostrar un mensaje escribíamos:

```
None
printf("Hola mundo");
```

En Python hacemos:

```
None
print("Hola mundo")
```

Como podemos observar, Python tiene una sintaxis más simple.

Características de Python

- Es un lenguaje de alto nivel.
- Tiene una sintaxis sencilla y fácil de leer.
- No necesitamos declarar el tipo de dato de una variable.
- Utiliza la indentación para organizar los bloques de código.

La indentación

En Python, la indentación consiste en dejar espacios al comienzo de una línea para indicar que una instrucción pertenece a un determinado bloque de código.

Por ejemplo:

```
None
edad = 20

if edad >= 18:
    print("Es mayor de edad")
```

Observemos que el print() tiene espacios al comienzo.

Esto indica que pertenece al bloque del if.

IMPORTANTE: En Python la indentación es obligatoria. A diferencia de C, no utilizamos llaves {} para delimitar los bloques de código.

Comentarios

Los comentarios son textos que escribimos dentro del código para explicar qué estamos haciendo.

La computadora los ignora, es decir, no los ejecuta.

Para escribir un comentario utilizamos #.

Ejemplo:

```
None
# Guardamos la edad del usuario
edad = 20

# Mostramos la edad
print(edad)
```

---

2. VARIABLES Y TIPOS DE DATOS

¿Qué es una variable?

Una variable es un espacio donde podemos almacenar información.

Como vimos anteriormente, podemos pensar una variable como una cajita que tiene:

- Una etiqueta (nombre).
- Un valor almacenado.

La diferencia principal con C es que en Python no necesitamos indicar previamente el tipo de dato.

En C escribíamos:

```
None
int edad = 16;
```

En Python simplemente hacemos:

```
None
edad = 16
```

Python reconoce automáticamente que estamos guardando un número entero.

Declaración y asignación

Para crear una variable utilizamos:

```
None
nombre_variable = valor
```

Ejemplos:

```
None
edad = 16
altura = 1.75
nombre = "Agustin"
aprobado = True
```

¿Qué está pasando?

Estamos creando cuatro variables y almacenando diferentes tipos de información.

Tipos de datos

Python tiene distintos tipos de datos dependiendo de la información que queremos almacenar.

int → Números enteros

Almacena números sin decimales.

```
None
edad = 18
cantidad = 25
temperatura = -5
```

float → Números decimales

Almacena números con decimales.

IMPORTANTE: Al igual que en C, los números decimales se escriben utilizando punto (.) y no coma (,).

```
None
altura = 1.75
precio = 2500.50
```

str → Cadenas de texto

Almacena texto, es decir, una secuencia de caracteres.

```
None
nombre = "Agustin"
apellido = "Castaneda"
```

Podemos utilizar comillas simples o dobles.

```
None
nombre = "Juan"
nombre = 'Juan'
```

Ambas formas son válidas.

bool → Valores booleanos

Almacena solamente dos posibilidades:

- True → Verdadero.
- False → Falso.

```
None
aprobado = True
mayor_edad = False
```

IMPORTANTE: En Python los valores booleanos comienzan con mayúscula: True y False.

¿Cómo saber el tipo de dato?

Podemos utilizar la función type().

Ejemplo:

```
None
edad = 18
nombre = "Juan"

print(type(edad))
print(type(nombre))
```

Resultado:

```
None
<class 'int'>
<class 'str'>
```

Python nos indica qué tipo de dato almacena cada variable.

Conversión de tipos de datos

Muchas veces necesitamos convertir un valor de un tipo de dato a otro.

Para eso utilizamos:

- int() → Convierte a entero.
- float() → Convierte a decimal.

- str() → Convierte a texto.

Ejemplo:

```
None
numero = "25"


numero = int(numero)
```

Antes, numero era un String (str).

Después de la conversión, pasa a ser un entero (int).

También podemos hacer:

```
None
precio = 1500
texto = str(precio)
```

Ahora texto contiene "1500".

---

3. ENTRADA Y SALIDA DE DATOS

La entrada y salida de datos permite que el usuario pueda interactuar con nuestro programa.

En Python utilizamos principalmente dos funciones:

print()

Permite mostrar información por pantalla (output).

Ejemplo:

```
None
print("Hola mundo")
```

Resultado:

```
None
Hola mundo
```

También podemos mostrar el contenido de una variable:

```
None
edad = 18


print(edad)
```

Resultado:

```
None
18
```

Podemos mostrar varios valores en un mismo print():

```
None
nombre = "Juan"
edad = 18

print(nombre, edad)
```

Resultado:

```
None
Juan 18
```

input()

Permite solicitar información al usuario (input).

Ejemplo:

```
None
nombre = input("Ingrese su nombre: ")

print(nombre)
```

El programa espera a que el usuario ingrese un dato.

Si escribe:

```
None
Agustin
```

La variable nombre almacenará ese texto.

Algo MUY importante sobre input()

La función input() siempre devuelve un String (str), aunque el usuario ingrese un número.

Por ejemplo:

```
None
edad = input("Ingrese su edad: ")
```

Si el usuario ingresa:

```
None
18
```

La variable edad contiene el texto "18", no el número entero 18.

Si queremos trabajar con ese valor como un número, debemos convertirlo.

Ejemplo:

```
None
edad = int(input("Ingrese su edad: "))
```

Ahora sí, edad almacena un entero.

Para números decimales utilizamos:

```
None
altura = float(input("Ingrese su altura: "))
```

Ejemplo completo

```
None
nombre = input("Ingrese su nombre: ")
edad = int(input("Ingrese su edad: "))

print("Hola", nombre)
print("Tu edad es:", edad)
```

De esta manera podemos recibir información, almacenarla en variables y mostrarla por pantalla.

Mostrar variables dentro de un texto

También podemos utilizar las llamadas f-strings.

Se escriben colocando una f antes de las comillas y las variables dentro de {}.

Ejemplo:

```
None
nombre = "Juan"
edad = 18

print(f"Hola {nombre}, tenes {edad} años")
```

Resultado:

```
None
Hola Juan, tenes 18 años
```

Es una forma práctica de combinar texto y variables.

---

4. OPERADORES ARITMÉTICOS

Los operadores aritméticos permiten realizar operaciones matemáticas.

Los principales son:

Operador	Operación	Ejemplo

+	Suma	3 + 5 = 8
-	Resta	3 - 5 = -2
*	Multiplicación	3 * 5 = 15
/	División	7 / 2 = 3.5
//	División entera	7 // 2 = 3
%	Módulo (resto)	7 % 2 = 1
**	Potencia	2 ** 3 = 8

Diferencia entre / y //

En Python tenemos dos formas de dividir.

Con / obtenemos el resultado de la división, incluyendo decimales.

```
None
resultado = 7 / 2
```

Resultado:

```
None
3.5
```

Con // obtenemos la división entera, descartando la parte decimal.

```
None
resultado = 7 // 2
```

Resultado:

```
None
3
```

Operador módulo (%)

El módulo permite obtener el resto de una división.

Ejemplo:

```
None
15 % 7
```

15 ÷ 7 = 2

Sobra 1.

Resultado:

```
None
1
```

Divisibilidad

Si queremos saber si un número es divisible por otro, podemos utilizar %.

```
None
numero % divisor == 0
```

Si el resto es 0, significa que el número es divisible.

Ejemplos:

```
None
14 % 7 == 0  # True
20 % 2 == 0  # True
15 % 7 == 0  # False
```

Incremento de variables

Muchas veces necesitamos aumentar o disminuir el valor de una variable.

Ejemplo:

```
None
edad = 15


edad = edad + 1
```

Ahora edad vale 16.

Forma abreviada

Python permite utilizar operadores abreviados:

```
None
edad += 1
```

Hace exactamente lo mismo que:

```
None
edad = edad + 1
```

Más ejemplos:

```
None
puntos += 10
saldo -= 500
precio *= 2
```

IMPORTANTE: A diferencia de C, Python no utiliza ++ ni --.

Para incrementar una variable en 1 utilizamos:

```
None
contador += 1
```

---

5. OPERADORES DE COMPARACIÓN Y LÓGICOS

Operadores de comparación

Los operadores de comparación permiten comparar dos valores.

El resultado de una comparación siempre será un booleano:

True o False.

Los principales son:

Operador	Significado
==	Igual
!=	Distinto
>	Mayor
<	Menor
>=	Mayor o igual
<=	Menor o igual

Ejemplos:

```
None
edad == 18
edad != 18
edad > 18
edad < 18
edad >= 18
edad <= 18
```

Por ejemplo:

```
None
edad = 20

print(edad >= 18)
```

Resultado:

```
None
True
```

Porque 20 es mayor o igual a 18.

Operadores lógicos

Los operadores lógicos permiten combinar o modificar condiciones.

En C utilizábamos &&, || y !.

En Python utilizamos palabras:

- and
- or
- not

AND (and)

Todas las condiciones deben cumplirse.

Ejemplo:

```
None
if edad >= 18 and tiene_entrada == True:
```

Para que toda la condición sea verdadera, ambas condiciones deben ser verdaderas.

OR (or)

Al menos una condición debe cumplirse.

Ejemplo:

```
None
if categoria == 2 or categoria == 7:
```

Si la categoría es 2 o es 7, la condición será verdadera.

NOT (not)

Invierte el resultado de una condición.

Ejemplo:

```
None
not True
```

Resultado:

```
None
False
```

Y:

```
None
not False
```

Resultado:

```
None
True
```

Recordá:

- and → Todas deben ser verdaderas.
- or → Al menos una debe ser verdadera.
- not → Invierte el resultado.

---

6. CONDICIONALES

Definición

Los condicionales permiten que un programa tome decisiones dependiendo de si una condición se cumple o no.

En Python utilizamos:

- if
- else
- elif

IF

Permite ejecutar instrucciones solamente si una condición es verdadera.

Sintaxis:

```
None
if condicion:
    instrucciones
```

Ejemplo:

```
None
edad = 20

if edad >= 18:
    print("Es mayor de edad")
```

Explicación:

El if evalúa si la edad es mayor o igual a 18.

La condición devuelve:

- True → Ejecuta el bloque de instrucciones.
- False → No ejecuta nada dentro del if.

IF - ELSE

El else permite indicar qué hacer cuando la condición del if no se cumple.

Ejemplo:

```
None
edad = 16

if edad >= 18:
    print("Es mayor")
else:
    print("Es menor")
```

Acá se lee:

Si la edad es mayor o igual a 18, mostramos "Es mayor".

Si no se cumple, mostramos "Es menor".

IF - ELIF - ELSE

Muchas veces necesitamos evaluar más de dos posibilidades.

Para eso utilizamos elif, que significa "si no se cumplió la condición anterior, probá con esta otra".

Ejemplo:

```
None
nota = 8

if nota >= 9:
    print("Excelente")
elif nota >= 6:
    print("Aprobado")
else:
    print("Desaprobado")
```

¿Qué está pasando?

1. Primero pregunta si la nota es mayor o igual a 9.
2. Si no se cumple, pregunta si es mayor o igual a 6.
3. Si ninguna condición se cumple, ejecuta el else.

Resultado:

```
None
Aprobado
```

IMPORTANTE: Python evalúa las condiciones de arriba hacia abajo. Cuando encuentra una condición verdadera, ejecuta ese bloque y no continúa evaluando los siguientes.

Condicionales anidados

Podemos colocar un if dentro de otro if.

Ejemplo:

```
None
edad = 20
tiene_entrada = True

if edad >= 18:
    if tiene_entrada:
        print("Puede ingresar")
```

Primero verifica si es mayor de edad.

Si se cumple, recién entonces verifica si tiene entrada.

Esto se llama condicional anidado.

---

7. BUCLES (WHILE, FOR)

7.1. BUCLE WHILE

Definición

El bucle while permite repetir instrucciones mientras una condición sea verdadera.

Es decir, el programa va a repetir un bloque de código hasta que la condición deje de cumplirse.

Sintaxis:

```
None
while condicion:
    instrucciones
```

Ejemplo:

```
None
i = 1

while i <= 5:
    print(i)
    i += 1
```

Salida:

```
None
1
2
3
4
5
```

Simulación paso a paso

La variable i comienza valiendo 1.

El while pregunta si i es menor o igual a 5.

Primera iteración

i = 1

¿1 <= 5? Sí.

Entonces:

- Imprime 1.
- Incrementa i en 1.

Ahora i = 2.

Segunda iteración

¿2 <= 5? Sí.

Imprime 2 y vuelve a incrementar.

El proceso continúa hasta que i vale 6.

En ese momento:

¿6 <= 5? No.

La condición es falsa y el ciclo termina.

IMPORTANTE: Evitar bucles infinitos

Un error muy común es olvidarnos de modificar la variable que controla el while.

Por ejemplo:

```
None
i = 1

while i <= 5:
    print(i)
```

Este ciclo nunca termina porque i siempre vale 1.

La condición siempre se cumple.

Por eso debemos asegurarnos de que en algún momento la condición deje de ser verdadera.

7.2. BUCLE FOR

Definición

El bucle for permite recorrer una secuencia de elementos o repetir instrucciones una determinada cantidad de veces.

En Python suele utilizarse junto con range() cuando conocemos cuántas veces queremos repetir algo.

Ejemplo:

```
None
for i in range(1, 6):
    print(i)
```

Salida:

```
None
1
2
3
4
5
```

¿Qué hace range()?

La función range() genera una secuencia de números.

Podemos utilizarla de distintas maneras:

```
None
range(5)
```

Genera:

```
None
0, 1, 2, 3, 4
```

```
None
range(1, 6)
```

Genera:

```
None
1, 2, 3, 4, 5
```

```
None
range(2, 11, 2)
```

Genera:

```
None
2, 4, 6, 8, 10
```

La sintaxis general es:

```
None
range(inicio, fin, paso)
```

- inicio → Desde dónde comienza.

- fin → Hasta dónde llega, sin incluirlo.
- paso → Cuánto aumenta o disminuye en cada iteración.

IMPORTANTE: El valor final nunca se incluye.

Simulación del FOR

Supongamos:

```
None
for i in range(1, 6):
    print(i)
```

El bucle realiza:

- Primera iteración: i = 1
- Segunda iteración: i = 2
- Tercera iteración: i = 3
- Cuarta iteración: i = 4
- Quinta iteración: i = 5

Cuando debería llegar a 6, termina porque el límite no está incluido.

Diferencia entre WHILE y FOR

WHILE

Se utiliza generalmente cuando no sabemos exactamente cuántas veces se repetirá el ciclo, sino que depende de una condición.

FOR

Se utiliza generalmente cuando queremos recorrer una secuencia o conocemos la cantidad de repeticiones.

BREAK y CONTINUE

También podemos controlar el comportamiento de los bucles.

break

Permite salir inmediatamente del ciclo.

Ejemplo:

```
None
for i in range(1, 10):
    if i == 5:
        break

    print(i)
```

Salida:

```
None
1
2
```

```
3
4
```

Cuando i llega a 5, el break termina el ciclo.

continue

Permite saltear la iteración actual y continuar con la siguiente.

Ejemplo:

```
None
for i in range(1, 6):
    if i == 3:
        continue

    print(i)
```

Salida:

```
None
1
2
4
5
```

Cuando i vale 3, no ejecuta el print() y pasa a la siguiente iteración.

---

8. FUNCIONES

Definición

Una función es un bloque de código que realiza una tarea específica.

Como vimos anteriormente, podemos pensar una función como una máquina:

Entrada → proceso → resultado.

En Python las funciones permiten organizar nuestro programa, reutilizar código y evitar repetir instrucciones.

Crear una función

Para crear una función utilizamos la palabra reservada def.

Sintaxis:

```
None
def nombre_funcion(parametros):
    instrucciones
```

Ejemplo:

```
None
def saludar():
    print("Hola mundo")
```

Esto crea una función llamada saludar.

Pero todavía no la ejecutamos.

Llamar una función

Para ejecutar una función debemos llamarla por su nombre.

```
None
saludar()
```

Resultado:

```
None
Hola mundo
```

Funciones con parámetros

Una función puede recibir información para trabajar con ella.

Esos datos se llaman parámetros.

Ejemplo:

```
None
def saludar(nombre):
    print("Hola", nombre)
```

Para utilizarla:

```
None
saludar("Agustin")
```

Resultado:

```
None
Hola Agustin
```

¿Qué está pasando?

- nombre es el parámetro de la función.
- "Agustin" es el argumento que enviamos al llamarla.

Funciones con varios parámetros

Podemos recibir más de un dato.

Ejemplo:

```
None
def sumar(numero1, numero2):
```

```
    resultado = numero1 + numero2
    print(resultado)
```

Llamada:

```
None
sumar(5, 3)
```

Resultado:

```
None
8
```

¿Qué significa RETURN?

Hasta ahora vimos funciones que muestran información utilizando print().

Pero muchas veces necesitamos que una función devuelva un resultado para poder utilizarlo en otra parte del programa.

Para eso utilizamos return.

Ejemplo:

```
None
def doble(numero):
    return numero * 2
```

Si hacemos:

```
None
resultado = doble(5)
```

La función calcula:

5 * 2 = 10

Y devuelve:

```
None
10
```

Entonces:

```
None
resultado = 10
```

Diferencia entre PRINT y RETURN

Esta diferencia es MUY importante.

print() → Muestra información por pantalla.

return → Devuelve un resultado para que podamos utilizarlo.

Ejemplo:

```
None
def sumar(a, b):
    return a + b

resultado = sumar(5, 3)

print(resultado)
```

Resultado:

```
None
8
```

La función devuelve el resultado de la suma y luego lo guardamos en una variable.

Funciones que devuelven booleanos

Una función también puede devolver True o False.

Ejemplo:

```
None
def es_par(numero):
    return numero % 2 == 0
```

Si hacemos:

```
None
resultado = es_par(10)
```

La función devuelve True, porque 10 es divisible por 2.

También podemos utilizarla directamente en un condicional:

```
None
if es_par(10):
    print("Es par")
```

Variables locales

Una variable creada dentro de una función pertenece a esa función.

Ejemplo:

```
None
def calcular():
    resultado = 10 + 5
    return resultado
```

La variable resultado existe dentro de la función.

No podemos utilizarla directamente desde afuera.

Esto se conoce como alcance o ámbito de una variable.

Ejemplo completo

```
None
def calcular_descuento(precio, porcentaje):
    descuento = precio * porcentaje / 100
    return precio - descuento

precio_final = calcular_descuento(1000, 20)

print(precio_final)
```

Resultado:

```
None
800.0
```

Acá combinamos:

- Funciones.
- Parámetros.
- Variables.
- Operadores aritméticos.
- Return.

---

9. STRINGS

¿Qué es un String?

Un String es una cadena de caracteres, es decir, un conjunto de letras, números, símbolos o espacios que representan texto.

En Python utilizamos el tipo de dato str.

Ejemplo:

```
None
nombre = "Agustin"
```

También podemos utilizar comillas simples:

```
None
nombre = 'Agustin'
```

Acceder a los caracteres

Un String está formado por caracteres que ocupan distintas posiciones.

Ejemplo:

```
None
palabra = "Hola"
```

Podemos imaginarlo así:

```
None
Caracter:  H   o   l   a
Posicion:  0   1   2   3
```

Para acceder a un carácter utilizamos su posición:

```
None
print(palabra[0])
```

Resultado:

```
None
H
```

IMPORTANTE: Al igual que en los vectores de C, las posiciones comienzan en 0.

Índices negativos

Python también permite acceder a los caracteres desde el final del String utilizando índices negativos.

Ejemplo:

```
None
palabra = "Hola"


print(palabra[-1])
```

Resultado:

```
None
a
```

Podemos pensarlo así:

```
None
Caracter:   H    o    l    a
Indice:     0    1    2    3
Indice:    -4   -3   -2   -1
```

Longitud de un String

Para conocer cuántos caracteres tiene una cadena utilizamos len().

Ejemplo:

```
None
palabra = "Hola"

longitud = len(palabra)

print(longitud)
```

Resultado:

```
None
4
```

Recorrer un String

Podemos recorrer todos los caracteres utilizando un for.

Ejemplo:

```
None
palabra = "Hola"

for caracter in palabra:
    print(caracter)
```

Salida:

```
None
H
o
l
a
```

También podemos recorrerlo utilizando posiciones:

```
None
palabra = "Hola"

for i in range(len(palabra)):
    print(palabra[i])
```

Ambas formas permiten acceder a cada carácter.

Unir Strings

Podemos unir dos cadenas utilizando el operador +.

Ejemplo:

```
None
nombre = "Juan"
apellido = "Perez"

nombre_completo = nombre + " " + apellido
```

Resultado:

```
None
Juan Perez
```

Repetir Strings

También podemos repetir una cadena utilizando *.

Ejemplo:

```
None
print("Hola " * 3)
```

Resultado:

```
None
Hola Hola Hola
```

Métodos importantes de Strings

Python incluye métodos que permiten trabajar con cadenas.

Método	¿Qué hace?
upper()	Convierte a mayúsculas
lower()	Convierte a minúsculas
title()	Coloca la primera letra de cada palabra en mayúscula
strip()	Elimina espacios al principio y al final
replace()	Reemplaza una parte del texto
count()	Cuenta apariciones
find()	Busca una subcadena
startswith()	Comprueba si comienza con un texto

endswith()	Comprueba si termina con un texto

Ejemplos:

```
None
nombre = "agustin"

print(nombre.upper())
```

Resultado:

```
None
AGUSTIN
```

```
None
texto = "Hola mundo"

print(texto.replace("mundo", "Python"))
```

Resultado:

```
None
Hola Python
```

```
None
palabra = "banana"

print(palabra.count("a"))
```

Resultado:

```
None
3
```

IMPORTANTE: Los Strings en Python son inmutables. Esto significa que no podemos modificar directamente un carácter de una cadena mediante su posición.

Por ejemplo, esto produce un error:

```
None
palabra = "hola"

palabra[0] = "H"
```

Si queremos modificar el texto, debemos generar una nueva cadena.

---

10. LISTAS

¿Qué es una lista?

Una lista es una estructura que permite almacenar varios elementos bajo un mismo nombre.

Podemos pensarla como un vector, similar a los que vimos en C.

La diferencia es que en Python las listas pueden cambiar de tamaño durante la ejecución y pueden almacenar distintos tipos de datos.

Ejemplo:

```
None
edades = [16, 18, 20, 15, 17]
```

Tenemos una lista llamada edades que almacena cinco números enteros.

También podemos crear listas con distintos tipos de datos:

```
None
datos = ["Juan", 18, 1.75, True]
```

Crear una lista

Utilizamos corchetes [].

```
None
numeros = [10, 20, 30, 40, 50]
```

También podemos crear una lista vacía:

```
None
numeros = []
```

Acceder a un elemento

Al igual que en los vectores, utilizamos la posición del elemento.

Ejemplo:

```
None
numeros = [10, 20, 30, 40, 50]


print(numeros[0])
```

Resultado:

```
None
10
```

Recordá: las posiciones comienzan en 0.

Modificar elementos

A diferencia de los Strings, las listas sí permiten modificar sus elementos directamente.
Ejemplo:

```
None
numeros = [10, 20, 30]

numeros[1] = 100
```

Ahora la lista contiene:

```
None
[10, 100, 30]
```

Agregar elementos

Python tiene distintos métodos para agregar elementos.

append()

Agrega un elemento al final de la lista.

```
None
numeros = [10, 20, 30]

numeros.append(40)
```

Resultado:

```
None
[10, 20, 30, 40]
```

insert()

Permite agregar un elemento en una posición determinada.

```
None
numeros = [10, 20, 30]

numeros.insert(1, 15)
```

Resultado:

```
None
[10, 15, 20, 30]
```

extend()

Permite agregar varios elementos de otra lista.

```
None
numeros = [10, 20]
```

```
numeros.extend([30, 40])
```

Resultado:

```
None
[10, 20, 30, 40]
```

Eliminar elementos

También podemos eliminar elementos de una lista.

remove()

Elimina la primera aparición de un valor determinado.

```
None
numeros = [10, 20, 30]

numeros.remove(20)
```

Resultado:

```
None
[10, 30]
```

pop()

Elimina un elemento utilizando su posición y devuelve el valor eliminado.

```
None
numeros = [10, 20, 30]

eliminado = numeros.pop(1)
```

Ahora:

```
None
numeros = [10, 30]
eliminado = 20
```

Si no indicamos una posición, pop() elimina el último elemento.

Longitud de una lista

Para conocer la cantidad de elementos utilizamos len().

```
None
numeros = [10, 20, 30]

print(len(numeros))
```

Resultado:

```
None
3
```

Recorrer una lista

Podemos recorrer todos sus elementos utilizando for.

Ejemplo:

```
None
numeros = [10, 20, 30, 40]

for numero in numeros:
    print(numero)
```

También podemos recorrerla mediante sus posiciones:

```
None
for i in range(len(numeros)):
    print(numeros[i])
```

Buscar elementos

Podemos utilizar el operador in para comprobar si un elemento existe dentro de una lista.

Ejemplo:

```
None
numeros = [10, 20, 30, 40]

if 20 in numeros:
    print("El numero existe")
```

Resultado:

```
None
El numero existe
```

Ordenar una lista

Python tiene el método sort() para ordenar los elementos.

De menor a mayor:

```
None
numeros = [8, 3, 10, 1, 5]

numeros.sort()
```

Resultado:

```
None
[1, 3, 5, 8, 10]
```

De mayor a menor:

```
None
numeros.sort(reverse=True)
```

Resultado:

```
None
[10, 8, 5, 3, 1]
```

Invertir una lista

Podemos utilizar reverse() para invertir el orden actual de los elementos.

```
None
numeros = [1, 2, 3, 4]


numeros.reverse()
```

Resultado:

```
None
[4, 3, 2, 1]
```

Listas anidadas

Una lista puede contener otras listas.

Esto permite representar información organizada en filas y columnas, de manera similar a una matriz.

Ejemplo:

```
None
matriz = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]
```

Para acceder a un elemento necesitamos dos posiciones:

```
None
print(matriz[1][2])
```

Resultado:

```
None
6
```

Porque estamos accediendo a la fila 1 y columna 2.

---

11. MANEJO DE ARCHIVOS

¿Qué es el manejo de archivos?

Hasta ahora, los datos que utilizamos se almacenaban en variables, listas o diccionarios mientras el programa estaba ejecutándose.

Pero cuando el programa termina, esa información normalmente se pierde.

Para conservar datos podemos utilizar archivos.

Por ejemplo:

- Guardar nombres de alumnos.
- Registrar notas.
- Almacenar información de clientes.
- Guardar resultados de un programa.

Python permite crear, leer y modificar archivos.

Abrir un archivo

Para trabajar con archivos utilizamos la función open().

Sintaxis:

```
None
archivo = open("datos.txt", "modo")
```

El modo indica qué queremos hacer con el archivo.

Los principales son:

Modo	Función
"r"	Leer un archivo
"w"	Escribir un archivo
"a"	Agregar información al final
"x"	Crear un archivo nuevo

Escribir en un archivo

Utilizamos el modo "w".

Ejemplo:

```
None
archivo = open("datos.txt", "w")

archivo.write("Hola mundo")

archivo.close()
```

¿Qué hace?

1. Abre el archivo datos.txt.
2. Escribe el texto "Hola mundo".
3. Cierra el archivo.

IMPORTANTE: El modo "w" puede reemplazar el contenido anterior del archivo si ya existe.

Leer un archivo

Utilizamos el modo "r".

Ejemplo:

```
None
archivo = open("datos.txt", "r")

contenido = archivo.read()

print(contenido)

archivo.close()
```

read() permite leer todo el contenido del archivo.

Agregar información

Si queremos agregar contenido sin borrar lo que ya existe, utilizamos el modo "a".

Ejemplo:

```
None
archivo = open("datos.txt", "a")

archivo.write("\nNuevo dato")

archivo.close()
```

El nuevo texto se agrega al final del archivo.

¿Qué es close()?

Cuando terminamos de trabajar con un archivo, debemos cerrarlo utilizando close().

Esto permite liberar los recursos utilizados por el archivo.

Utilizar WITH

Python tiene una forma más práctica y segura de trabajar con archivos: with.

Ejemplo:

```
None
with open("datos.txt", "r") as archivo:
    contenido = archivo.read()
    print(contenido)
```

Cuando termina el bloque with, Python cierra automáticamente el archivo.

Por eso generalmente se recomienda utilizar esta forma.

---

12. ORGANIZACIÓN Y BUENAS PRÁCTICAS

A medida que nuestros programas crecen, es importante organizar correctamente el código.

No alcanza con que el programa funcione. También necesitamos que sea fácil de leer, entender y modificar.

Utilizar nombres descriptivos

Es importante elegir nombres que expliquen qué información almacena una variable o qué tarea realiza una función.

Por ejemplo, es mejor:

```
None
edad_alumno = 18
```

Que:

```
None
x = 18
```

Lo mismo ocurre con las funciones.

Es mejor:

```
None
def calcular_promedio():
```

Que:

```
None
def funcion1():
```

Separar el programa en funciones

Cuando un programa tiene muchas instrucciones, conviene dividirlo en funciones más pequeñas.

Ejemplo:

```
None
def pedir_datos():
    nombre = input("Ingrese su nombre: ")
    edad = int(input("Ingrese su edad: "))
    return nombre, edad


def mostrar_datos(nombre, edad):
    print("Nombre:", nombre)
    print("Edad:", edad)
```

De esta manera cada función tiene una responsabilidad específica.

Función principal: main()

Podemos organizar el programa utilizando una función principal llamada main().

Ejemplo:

```
None
def saludar():
    print("Hola mundo")


def main():
    saludar()


main()
```

La función main() permite centralizar el inicio y la organización general del programa.

Módulos e IMPORT

Python permite dividir nuestro código en distintos archivos.

Cada archivo de Python puede funcionar como un módulo.

También podemos utilizar módulos que ya vienen preparados con el lenguaje.

Para incorporarlos utilizamos import.

Ejemplo:

```
None
import math

resultado = math.sqrt(25)

print(resultado)
```

Resultado:

```
None
5.0
```

En este caso utilizamos el módulo math, que contiene funciones matemáticas.

Otros módulos conocidos son:

- random → Generar valores aleatorios.
- math → Operaciones matemáticas.
- datetime → Trabajar con fechas y horarios.

Resolver un problema paso a paso

Cuando nos enfrentamos a un ejercicio de programación, no debemos comenzar escribiendo código sin antes entender qué nos están pidiendo.

Una buena estrategia es:

1. **Comprender el problema:** ¿Qué tenemos que resolver?
2. **Identificar los datos de entrada:** ¿Qué información necesitamos?
3. **Identificar el resultado:** ¿Qué debemos obtener?
4. **Pensar el procedimiento:** ¿Qué operaciones o decisiones necesitamos?
5. **Dividir el problema:** ¿Podemos resolverlo mediante funciones?
6. **Escribir el código:** Traducir nuestra solución al lenguaje Python.
7. **Probar el programa:** Verificar que funcione correctamente con distintos valores.

La programación no consiste solamente en conocer las instrucciones de un lenguaje.

Lo más importante es aprender a pensar cómo resolver un problema y después traducir esa solución a código.

RESUMEN DE SÍMBOLOS Y FUNCIONES IMPORTANTES

Elemento	¿Para qué sirve?
=	Asignar un valor
==	Comparar si dos valores son iguales
!=	Comparar si son distintos
+	Sumar
-	Restar
*	Multiplicar
/	Dividir

//	División entera
%	Obtener el resto
**	Potencia
+=	Incrementar o sumar a una variable
and	Unir condiciones que deben cumplirse todas
or	Unir condiciones donde alcanza con una verdadera
not	Invertir un valor lógico
if	Evaluar una condición
elif	Evaluar otra condición
else	Ejecutar una alternativa
while	Repetir mientras una condición sea verdadera
for	Recorrer una secuencia o repetir instrucciones
range()	Generar una secuencia de números
break	Salir de un bucle
continue	Saltear una iteración
def	Crear una función
return	Devolver un resultado
print()	Mostrar información
input()	Solicitar información
type()	Consultar el tipo de dato
int()	Convertir a entero

float()	Convertir a decimal
str()	Convertir a texto
bool()	Convertir a booleano
len()	Obtener la cantidad de elementos
append()	Agregar un elemento al final de una lista
insert()	Agregar un elemento en una posición
remove()	Eliminar un elemento por su valor
pop()	Eliminar un elemento por su posición
sort()	Ordenar una lista
open()	Abrir un archivo
import	Incorporar un módulo
try / except	Controlar errores